

M09 Review / Repair - Teaching RC2

OPEN ONLY AFTER SAVED ATTEMPT

Use surgically

Read only the competency you missed. Close this file and solve a changed retry. Reading review is not mastery.

P01 repair - ETL vs ELT and transformation boundaries

Transformation is the stage where raw meaning becomes business-ready data. Where it runs affects raw preservation, compute cost, security and recovery.

Core repair rule: ETL = Extract -> Transform -> Load; ELT = Extract -> Load -> Transform.

Proof: governance -> where are rules/versioning strongest?

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P02 repair - Raw, staging, clean/intermediate and serving contracts

Without layer contracts teams rewrite the same rules in many places and cannot tell which dataset is trustworthy.

Core repair rule: Raw/landing preserves source representation + ingestion metadata.

Proof: serving -> one date-region aggregate

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P03 repair - Deterministic transformations and production SQL style

A deterministic pipeline is reproducible: same input + code/config yields the same result. Hidden current-time/random/order dependencies make debugging and reruns unreliable.

Core repair rule: Avoid SELECT * in production transformation contracts; list columns so upstream additions do not silently change output.

Proof: ordering when selecting one row -> stable tie-breaker

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P04 repair - Deduplication and key integrity

Duplicates can be exact retries, legitimate multiple events or conflicting versions. `drop_duplicates()` is not a business rule.

Core repair rule: First define target grain/business key.

Proof: row accounting -> $\text{input} = \text{kept} + \text{removed/quarantined}$

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P05 repair - Schema checks and data contracts

Transformations are built on assumptions. A renamed/missing/type-changed column can create wrong output even when parsing succeeds.

Core repair rule: Check required columns and unexpected/optional policy before transformation.

Proof: renamed field -> breaking until contract update

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P06 repair - Source-to-target reconciliation

A pipeline can run without exceptions and still lose rows or multiply money through a join. Reconciliation is correctness evidence.

Core repair rule: Use row-count reconciliation only when grain/filter semantics make it meaningful.

Proof: reject reasons -> explained exclusions

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P07 repair - Incremental processing and safe reruns

Incremental pipelines are stateful. Correctness depends on what is already processed and how the same window behaves on retry.

Core repair rule: Incremental boundary may be source file identity, watermark, partition/date or CDC position.

Proof: late/backfill -> explicit window/policy

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P08 repair - Failure behavior, retries, quarantine and atomic state

A robust batch pipeline is defined by what happens when it fails halfway, not only by the happy path.

Core repair rule: Classify failures: transient infrastructure vs deterministic data/code/contract.

Proof: success -> publish + state/run commit

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P09 repair - Lineage, run metadata and auditability

When a number is questioned, you need to answer which input/run/code created it and how to reproduce it.

Core repair rule: Record run_id, start/end/status and source input identities/hashes/windows.

Proof: where output? -> table/file/partition/version

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P10 repair - Build and defend a reliable batch pipeline

The pipeline practical is where individual quality concepts become one correctness argument.

Core repair rule: Start by documenting input/window and layer contracts.

Proof: lineage -> run/input/code/output

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.